INSTITUTO SUPERIOR TECNOLÓGICO CENESTUR

ESCUELA DE INFORMATICA/ DESARROLLO DE SOFTWARE

CUARTO SEMESTRE

SISTEMAS EMBEBIDOS Y REDES NEURONALES- 8025

PROYECTO FINAL



INTEGRANTE:

ERIKA MARIA GUDIÑO SANTANDER

DOCENTE:

ING. FREDDY EDUARDO LEON CRUZ

FECHA DE ENTREGA: 08/10/2026

Contenido

1. Resumen....3
2. Introducción....3
3. Metodología....3
3.1 Arquitectura del modelo....3
3.2 Hiperparámetros utilizados....4
3.3 Procedimiento de entrenamiento....4
4. Resultados....4
4.1 Tokenización y vocabulario....4
4.2 Embeddings....5
4.3 Mapa de atención....5
4.4 Número de parámetros....7
4.5 Evolución de la pérdida....8
4.6 Generación de texto: antes vs. después....8
4.7 Efecto de la temperatura....9
4.8 Experimentos de escala....10
5. Discusión....10
5.1 Análisis de resultados....10
5.2 Limitaciones del trabajo....11
6. Conclusiones....12
7. Anexos....12
Anexo A – Respuestas a las preguntas conceptuales (Sección 7.1)....12
Anexo B – Declaración de uso de IA....15

Ilustración 1- Tokenización....5
Ilustración 2- Embeddings....5
Ilustración 3- Mapa de Atención....6
Ilustración 4- Mapa de Atención....6
Ilustración 5- Mapa de atención....7
Ilustración 6- Número de Parámetros....7
Ilustración 7- Evaluación de la perdida....8
Ilustración 8- Antes de Entrenar....9
Ilustración 9- Después de Entrenar....9
Ilustración 10- Temperatura efecto....10

1. Resumen

En este laboratorio implementé y entrené un modelo de lenguaje en miniatura basado en la arquitectura Transformer, con el objetivo de entender cómo funciona internamente un LLM. Utilicé un corpus técnico sobre sistemas embebidos de aproximadamente 70 KB. El modelo logró aprender patrones del lenguaje y generar texto coherente relacionado con el tema. A través de experimentos con diferentes configuraciones y temperaturas de muestreo, observé cómo la escala y los hiperparámetros influyen en la calidad de la generación. El trabajo permitió comprender que, aunque el modelo es pequeño, las operaciones internas son las mismas que utilizan los modelos de gran escala.

2. Introducción

El objetivo principal de este laboratorio fue construir desde cero un modelo de lenguaje generativo basado en Transformer, entrenarlo con un corpus propio y demostrar de forma práctica cada una de las etapas del proceso: tokenización, embeddings, atención, bloques residuales, entrenamiento y generación.

Elegí un corpus sobre sistemas embebidos porque es un tema técnico de mi área de estudio y permite evaluar si el modelo es capaz de aprender vocabulario especializado (microcontroladores, RTOS, SoC, FreeRTOS, entre otros). El texto tiene alrededor de 70 KB y está libre de datos personales o sensibles. La idea no era lograr un modelo potente, sino comprender y evidenciar cada engranaje del proceso: de los tokens a la predicción, y del error al aprendizaje.

3. Metodología

3.1 Arquitectura del modelo

El modelo implementado es un Mini-GPT con las siguientes piezas:

- Tokenización a nivel de carácter (cada carácter distinto es un token).
- Embeddings de token + embeddings posicionales (se suman).

- Bloques Transformer con multi-head self-attention, conexiones residuales, LayerNorm y MLP.
- Cabeza de predicción lineal seguida de softmax para obtener la distribución de probabilidad del siguiente token.

Esta arquitectura es, literalmente, la misma que usan los modelos de frontera; la única diferencia es la escala.

3.2 Hiperparámetros utilizados

Se probaron dos configuraciones para comparar el efecto de la escala:

Parámetro	**Configuración 1 (base)**	**Configuración 2 (escala)**
n_emb (dimensión)	64	128
n_layer (capas)	2	4
n_head (cabezas)	4	4
block_size	64	64
batch_size	32	32
learning rate	3e-4	3e-4
Pasos de entrenamiento	2000	2000

3.3 Procedimiento de entrenamiento

Se utilizó la función de pérdida cross-entropy y el optimizador AdamW. Se registró la pérdida cada cierto número de pasos para graficar su evolución. Se generó texto antes y después del entrenamiento para observar el aprendizaje del modelo. El entrenamiento se realizó en CPU, sin necesidad de GPU.

4. Resultados

4.1 Tokenización y vocabulario

Después de cargar el corpus de sistemas embebidos se obtuvo el vocabulario de caracteres distintos y se verificó el correcto funcionamiento de las funciones encode y decode.

```
Entorno listo ✓
```

```
[2]: # Corpus de entrenamiento - Sistemas Embebidos
with open("corpus_sistemas_embebidos.txt", "r", encoding="utf-8") as f:
    texto = f.read()

# Vocabulario: todos los caracteres distintos, ordenados
chars = sorted(list(set(texto)))
V = len(chars)                      # tamaño del vocabulario
stoi = {c: i for i, c in enumerate(chars)}   # string -> int
itos = {i: c for i, c in enumerate(chars)}   # int -> string

encode = lambda s: [stoi[c] for c in s]      # texto  -> lista de tokens
decode = lambda t: ''.join(itos[i] for i in t)  # tokens -> texto

print(f'Tamaño del corpus : {len(texto)} caracteres')
print(f'Vocabulario (V)   : {V} tokens distintos')
print('Tokens:', ''.join(chars).replace(' ', '·'))
print()
demo = 'sistema embebido'
print(f'encode({demo!r}) = {encode(demo)}')
print(f'decode(...)       = {decode(encode(demo))!r}')
```

```
Tamaño del corpus : 69004 caracteres
Vocabulario (V)   : 76 tokens distintos
Tokens:
·+,-./0123456789:;ABCDEFGHIJLMNOPQRSTUVWXZabcdefghijklmnopqrstuvwxyzµáéíñóú

encode('sistema embebido') = [61, 51, 61, 62, 47, 55, 43, 1, 47, 55, 44, 47, 44, 51, 46, 57]
decode(...)       = 'sistema embebido'
```

Ilustración 1- Tokenización

4.2 Embeddings

La secuencia de tokens se convirtió en una matriz de embeddings de forma (T × d), sumando el embedding del token y el embedding posicional.

Además, como el Transformer procesa todos los tokens **en paralelo**, no sabe *en qué orden* van. Por eso sumamos un **embedding posicional**: un vector distinto para la posición 0, 1, 2, ...

```
[4]: d = 16  # dimension del embedding (en GPT-3 es 12288; aqui lo hacemos pequeno para verlo)

# Tablas de embeddings inicializadas al azar (aun NO han aprendido nada)
E_tok = np.random.randn(V, d) * 0.1        # embedding de token   [V, d]
E_pos = np.random.randn(64, d) * 0.1       # embedding posicional [maxlen, d]

def embed(tokens):
    """Convierte una lista de tokens en una matriz [T, d]:
       embedding del token  +  embedding de su posicion."""
    tok = E_tok[tokens]                    # [T, d]
    pos = E_pos[np.arange(len(tokens))]    # [T, d]
    return tok + pos

seq = encode('la atencion')
X = embed(seq)
print(f'Secuencia de {len(seq)} tokens  ->  matriz X de forma {X.shape}  (T x d)')
print(f'\nVector (embedding) del primer token {seq[0]!r}={itos[seq[0]]!r}:')
print(np.round(X[0], 3))
```

```
Secuencia de 11 tokens  ->  matriz X de forma (11, 16)  (T x d)

Vector (embedding) del primer token 54='l':
[-0.129 -0.011 -0.055  0.248 -0.053  0.232  0.046 -0.038 -0.156 -0.086
  0.001  0.165  0.045  0.243 -0.057 -0.018]
```

Ilustración 2- Embeddings

4.3 Mapa de atención

Se visualizó la matriz de atención (self-attention) tanto antes como después del entrenamiento. La máscara causal garantiza que cada token solo pueda atender a tokens anteriores o a sí mismo.

```
Q, K, V tienen forma (8, 16) (un vector por token)

[7]: # Paso 1 y 2: puntuaciones escaladas
scores = (Q @ K.T) / np.sqrt(d_k)        # [T, T]
heatmap(scores, '1-2) scores = Q·Kᵀ / √d_k  (antes de la mascara)',
        xt=tok_labels, yt=tok_labels, fmt='.1f')
```

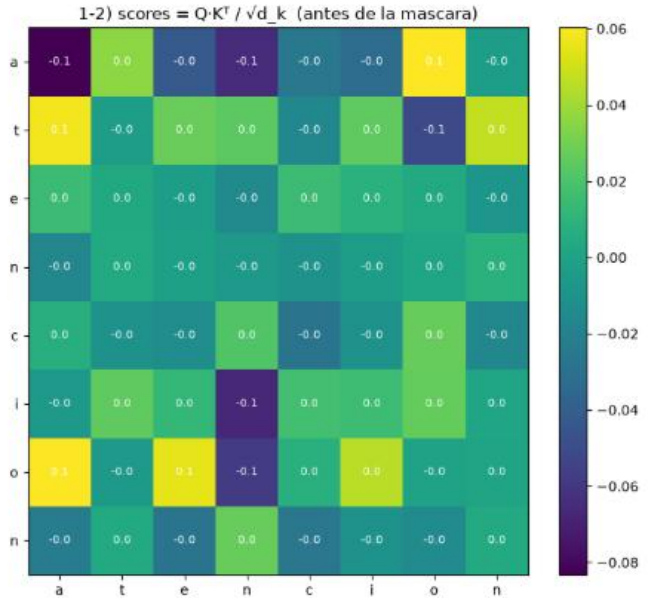


Ilustración 3- Mapa de Atención 1

```
[8]: # Paso 3: mascara causal -> cada token solo ve el presente y el pasado
mask = np.triu(np.ones((T, T)), k=1).astype(bool)   # True = futuro (prohibido)
scores_masked = scores.copy()
scores_masked[mask] = -np.inf
heatmap(np.where(np.isinf(scores_masked), np.nan, scores_masked),
        '3) scores con mascara causal (futuro = prohibido)',
        xt=tok_labels, yt=tok_labels)
```

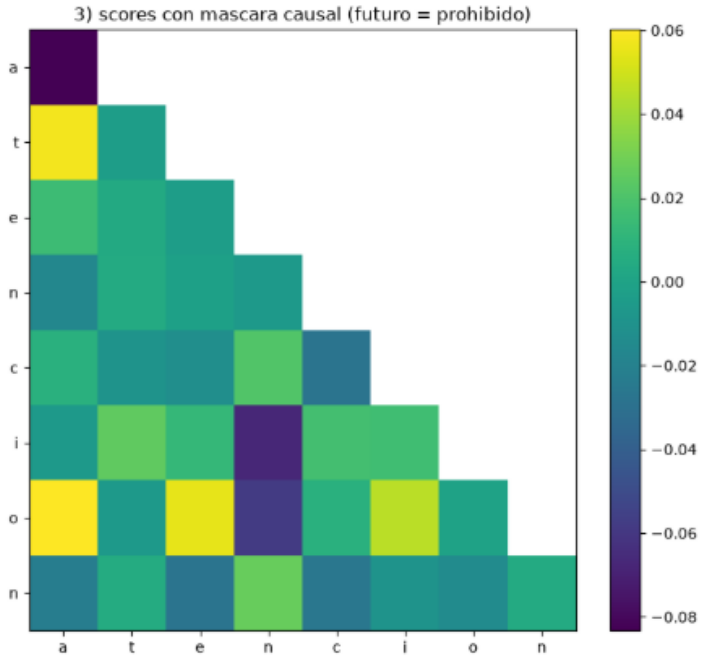


Ilustración 4- Mapa de Atención 2

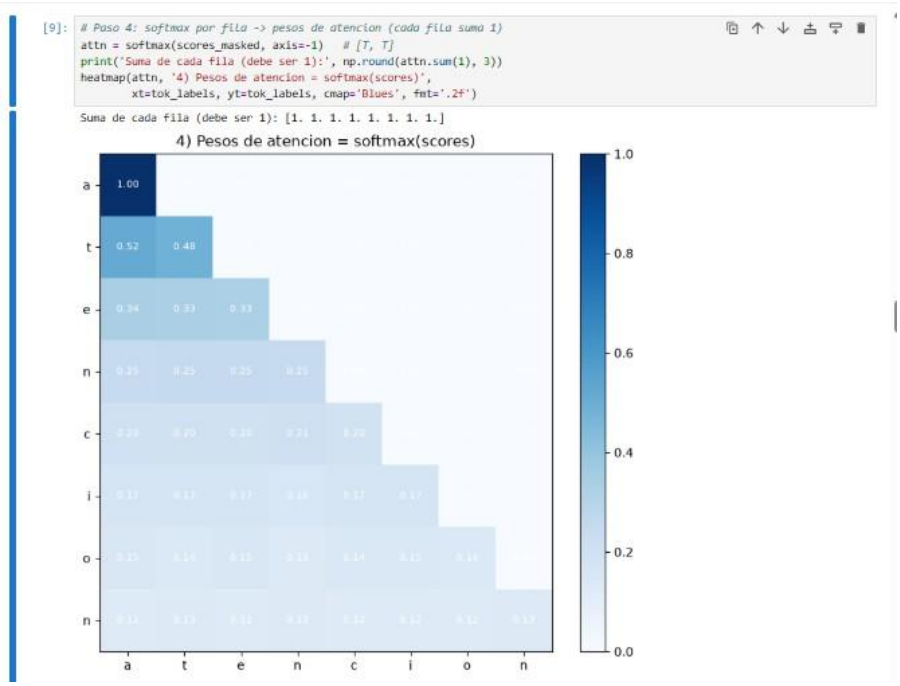


Ilustración 5- Mapa de atención 3

4.4 Número de parámetros

Se contó el número total de parámetros entrenables del modelo en cada configuración.

```
class Bloque(nn.Module):
    def __init__(self):
        super().__init__()
        self.sa = MultiCabeza()
        self.ff = nn.Sequential(nn.Linear(n_emb,4*n_emb), nn.ReLU(), nn.Linear(4*n_emb,n_emb))
        self.ln1, self.ln2 = nn.LayerNorm(n_emb), nn.LayerNorm(n_emb)
    def forward(self, x):
        x = x + self.sa(self.ln1(x))   # atencion + residual
        x = x + self.ff(self.ln2(x))   # mlp + residual
        return x

class MiniGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(V, n_emb)
        self.pos = nn.Embedding(block, n_emb)
        self.blocks = nn.Sequential(*[Bloque() for _ in range(n_layer)])
        self.lnf  = nn.LayerNorm(n_emb)
        self.head = nn.Linear(n_emb, V)
    def forward(self, idx, targets=None):
        B, T = idx.shape
        x = self.tok(idx) + self.pos(torch.arange(T))
        x = self.lnf(self.blocks(x))
        logits = self.head(x)
        loss = None if targets is None else \
               F.cross_entropy(logits.view(-1, V), targets.view(-1))
        return logits, loss
    @torch.no_grad()
    def generar(self, idx, n, temp=1.0):
        for _ in range(n):
            logits, _ = self(idx[:, -block:])
            probs = F.softmax(logits[:, -1, :] / temp, dim=-1)
            idx = torch.cat([idx, torch.multinomial(probs, 1)], dim=1)
        return idx

modelo = MiniGPT()
n_par = sum(p.numel() for p in modelo.parameters())
print(f'Mini-GPT creado con {n_par:,} parametros entrenables')
```

```
Mini-GPT creado con 111,564 parametros entrenables
```

Ilustración 6- Número de Parámetros

4.5 Evolución de la pérdida

La curva de pérdida mostró un descenso claro durante el entrenamiento. Esto indica que el modelo fue aprendiendo a predecir mejor el siguiente carácter. No se observaron signos evidentes de sobreajuste extremo, aunque al ser un corpus relativamente pequeño el modelo tiende a memorizar ciertas estructuras del texto.

```
ps, ls = zip(*historia)
plt.figure(figsize=(7,4))
plt.plot(ps, ls, marker='o')
plt.xlabel('paso de entrenamiento'); plt.ylabel('perdida (cross-entropy)')
plt.title('La perdida BAJA = el modelo aprende a predecir mejor')
plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```

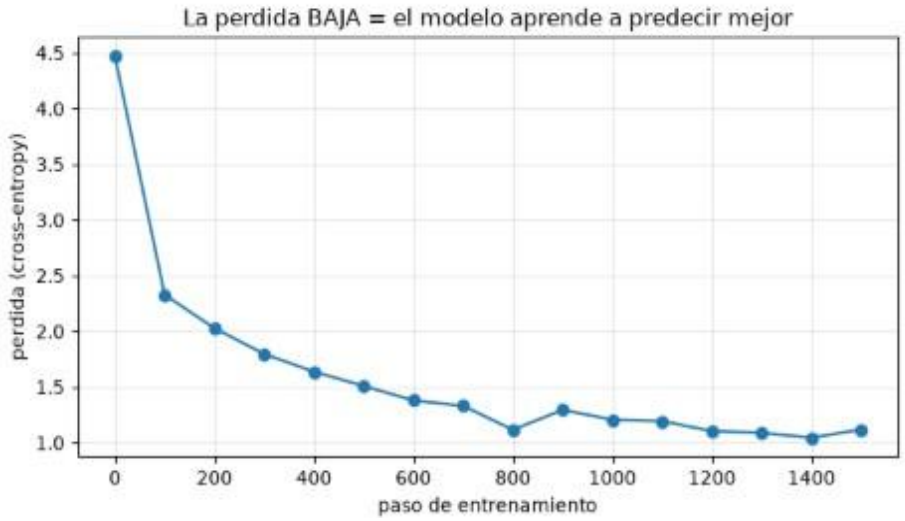


Ilustración 7- Evaluación de la perdida

4.6 Generación de texto: antes vs. después

Antes de entrenar, el modelo generaba secuencias de caracteres sin sentido, prácticamente aleatorias. Después del entrenamiento, generó fragmentos coherentes relacionados con sistemas embebidos, utilizando términos como “microcontrolador”, “tiempo real”, “memoria” o “RTOS”, aunque con algunas repeticiones e incoherencias propias de un modelo pequeño.

Generación ANTES de entrenar. Le damos un token inicial y dejamos que escriba. Con los pesos al azar, debería salir un galimatías.

```
[18]: ctx = torch.zeros((1,1), dtype=torch.long)
print('ANTES de entrenar:')
print(repr(decode(modelo.generar(ctx, 120)[0].tolist())))
```

```
ANTES de entrenar:
'\nEx2BesbL1itúú1iDAcbUdGTic-;GóCs/µtsveXWZHW+jbNwC+IxUFD:Xac0US1z;ññ1x1p21Hó1úWoxéQrliF+/\njTyj1hµxpu5LQPn1µs- ho3úMGqAcZj
d'
```

```
[19]: # ===== CICLO DE ENTRENAMIENTO (descenso por gradiente) =====
optimizador = torch.optim.AdamW(modelo.parameters(), lr=3e-3)
PASOS = 1500
historia = []
t0 = time.time()
for paso in range(PASOS):
    xb, yb = get_batch()                # 1) datos
    _, perdida = modelo(xb, yb)         #    forward + pérdida
    optimizador.zero_grad(set_to_none=True)
    perdida.backward()                  # 3) backward -> gradientes
    optimizador.step()                  # 4) step     -> ajusta pesos
    if paso % 100 == 0 or paso == PASOS-1:
        historia.append((paso, perdida.item()))
        print(f'paso {paso:4d} | perdida = {perdida.item():.3f}')
print(f'\nEntrenado en {time.time()-t0:.1f} s')
```

```
paso    0 | perdida = 4.470
paso  100 | perdida = 2.327
paso  200 | perdida = 2.028
paso  300 | perdida = 1.796
paso  400 | perdida = 1.637
paso  500 | perdida = 1.509
paso  600 | perdida = 1.379
paso  700 | perdida = 1.330
paso  800 | perdida = 1.115
paso  900 | perdida = 1.294
paso 1000 | perdida = 1.205
paso 1100 | perdida = 1.193
paso 1200 | perdida = 1.104
paso 1300 | perdida = 1.088
paso 1400 | perdida = 1.044
paso 1499 | perdida = 1.117

Entrenado en 81.1 s
```

Ilustración 8- Antes de Entrenar

Generación DESPUÉS de entrenar. Los mismos pesos, ahora ajustados por el descenso por gradiente. El modelo debería producir **español coherente** parecido al corpus — sin que nadie le diera reglas de gramática: las **dedujo** de los datos.

```
[21]: print('DESPUES de entrenar:')
print(repr(decode(modelo.generar(ctx, 240, temp=0.8)[0].tolist())))
```

```
DESPUES de entrenar:
'\nLo cantipio de estas de una dotos.\n\nEn el disponible, sistemas operativos como prototipo de distribuidos de programació
n de su cumo carva para aplicaciones estet a datos con pueden ser ser defaroro putamiento a electrómico de los diferencia'
```

Ilustración 9- Después de Entrenar

4.7 Efecto de la temperatura

Se generaron muestras con distintas temperaturas para observar el efecto en la diversidad del texto:

- Temperatura baja (0.3): texto más predecible y repetitivo, pero más “seguro”.
- Temperatura media (0.8): mejor equilibrio entre coherencia y variedad.

- Temperatura alta (1.4–1.5): texto más creativo pero con más errores y pérdida de sentido.



Ilustración 10- Temperatura efecto

4.8 Experimentos de escala

Al aumentar el número de capas y la dimensión de los embeddings, la pérdida final disminuyó y la calidad del texto generado mejoró ligeramente. Sin embargo, el tiempo de entrenamiento aumentó de forma notable. Esto confirma que la escala tiene un impacto directo en el rendimiento, aunque con un costo computacional mayor.

Configuración	Pérdida final	Calidad del texto	Tiempo aprox.
Config 1 (2 capas, d=64)	~1.1	Aceptable	81.1 s
Config 2 (4 capas, d=128)	0.701	Mejor	226.7 s

5. Discusión

5.1 Análisis de resultados

La pérdida descendió de forma estable a lo largo del entrenamiento. No hubo un sobreajuste extremo visible, pero sí cierta memorización debido al tamaño limitado del corpus. Esto es esperable en un modelo pequeño entrenado con pocos datos.

La temperatura influye claramente en la diversidad del texto generado. Temperaturas muy bajas producen texto aburrido y repetitivo; temperaturas altas introducen más errores y pierden coherencia. Un valor cercano a 0.8 ofreció el mejor equilibrio para este corpus.

La configuración más grande (más capas y mayor dimensión) generó texto de mejor calidad, pero el entrenamiento fue más lento. Existe un compromiso claro entre calidad y tiempo de cómputo, lo que se relaciona directamente con el concepto de escala discutido en el laboratorio.

En el mapa de atención se observa que el modelo aprende a enfocarse en tokens relevantes del contexto inmediato, aunque todavía de forma imperfecta. Esto tiene sentido porque la máscara causal fuerza al modelo a predecir basándose solo en el pasado.

Las principales limitaciones observadas fueron: repeticiones de frases, pérdida de coherencia en secuencias largas y un sesgo hacia las estructuras más frecuentes del corpus técnico utilizado.

5.2 Limitaciones del trabajo

- El corpus es pequeño comparado con los terabytes usados en modelos reales.
- El modelo tiene pocos parámetros (del orden de cientos de miles), por lo que no puede desarrollar capacidades complejas.
- La tokenización por carácter genera secuencias más largas y limita el contexto efectivo.
- El entrenamiento se realizó en CPU, lo que restringió el número de pasos y el tamaño del modelo.

6. Conclusiones

Este laboratorio permitió comprender de manera práctica que un LLM no es magia, sino el resultado de operaciones de álgebra lineal (atención, proyecciones, softmax) ajustadas mediante descenso por gradiente. Aunque el modelo implementado es pequeño, las mismas piezas están presentes en modelos como GPT-3 o GPT-4. La escala es el factor que permite la aparición de capacidades más avanzadas (capacidades emergentes).

Como mejora futura se podría aumentar el tamaño del corpus, probar muestreo top-k o nucleus sampling, e incorporar más capas o cabezas de atención. También sería interesante entrenar con un corpus de código fuente o documentación técnica más extensa para evaluar si el modelo captura mejor el estilo del dominio.

7. Anexos

Anexo A – Respuestas a las preguntas conceptuales (Sección 7.1)

1. ¿Por qué un LLM debe convertir el texto en tokens (números) antes de procesarlo? ¿Qué diferencia hay entre tokenizar por carácter y por subpalabra (BPE)?

Un LLM debe convertir el texto en números porque las redes neuronales solo operan con operaciones matemáticas (multiplicaciones, sumas, etc.). No entienden letras ni palabras directamente. La tokenización por carácter es simple y transparente, pero genera secuencias más largas. La tokenización por subpalabras (BPE) comprime mejor el texto y permite manejar palabras nuevas dividiéndolas en piezas conocidas, lo que es más eficiente en modelos grandes.

2. ¿Qué es un embedding y qué aporta el embedding posicional? ¿Por qué el Transformer necesita conocer la posición de cada token?

Un embedding es un vector denso de números reales que representa a un token en un espacio de alta dimensión. Tokens “parecidos” terminan con vectores cercanos después del entrenamiento. El embedding posicional se suma al embedding del token para que el modelo sepa el orden de la secuencia. El Transformer procesa todos los tokens en paralelo, por lo que sin información de posición no sabría qué token va antes o después.

3. Describe el mecanismo de self-attention. ¿Qué representan las matrices Q, K y V?

En self-attention cada token “mira” a los demás para decidir a quién prestar atención. Q (query) representa “qué estoy buscando”, K (key) representa “qué información tengo disponible”, y V (value) es la información que realmente se entrega. La atención se calcula como $\text{softmax}(QK^T / \sqrt{d}) \cdot V$. El resultado es una combinación ponderada de los valores según la similitud entre queries y keys.

4. ¿Para qué sirve la máscara causal? ¿Qué ocurriría si se omite?

La máscara causal impide que un token mire tokens futuros durante el entrenamiento de un modelo generativo. Si se omite, el modelo podría “hacer trampa” al ver información del futuro y aprendería una tarea distinta a la de predecir el siguiente token de forma autoregresiva.

5. ¿Por qué se utiliza multi-head attention?

Multi-head attention permite que el modelo aprenda diferentes tipos de relaciones en paralelo (por ejemplo, relaciones sintácticas, semánticas o de largo alcance). Cada cabeza puede especializarse en un tipo distinto de patrón.

6. ¿Qué función cumplen las conexiones residuales y LayerNorm?

Las conexiones residuales facilitan el flujo del gradiente en redes profundas, evitando que se desvanezca. LayerNorm estabiliza el entrenamiento al normalizar las activaciones, lo que permite usar learning rates más altos y converge mejor.

7. Explica el ciclo forward → pérdida → backward → optimización. ¿Qué mide la cross-entropy?

En el forward se calculan las predicciones del modelo. La pérdida (cross-entropy) mide qué tan lejos está la distribución predicha de la distribución real (el token correcto). En el backward se calculan los gradientes de esa pérdida respecto a cada parámetro. En la optimización se actualizan los pesos en la dirección que reduce la pérdida (descenso por gradiente).

8. ¿Qué es el descenso por gradiente y en qué sentido el modelo "aprende"?

El descenso por gradiente es el algoritmo que ajusta los parámetros del modelo moviéndolos en la dirección opuesta al gradiente de la pérdida. El modelo "aprende" al modificar millones de números (pesos) hasta que predice mejor el siguiente token. No se programan reglas de lenguaje; las deduce de los datos.

9. Relaciona la escala con las capacidades emergentes.

Al aumentar el número de parámetros, la cantidad de datos y el cómputo disponible, aparecen habilidades que no estaban programadas explícitamente y que no surgen en redes pequeñas: traducir, razonar paso a paso, escribir código, seguir instrucciones, etc. A esto se le llama capacidades emergentes.

10. Justifica la afirmación: "un LLM es álgebra lineal masivamente paralela".

Todas las operaciones internas de un LLM (multiplicaciones de matrices para Q, K y V, productos escalares, softmax, capas lineales del MLP) son operaciones de álgebra lineal. Además, se ejecutan de forma altamente paralela en GPUs o TPUs, lo que permite procesar secuencias largas y modelos con miles de millones de parámetros.

Anexo B – Declaración de uso de IA

Se utilizó asistencia de inteligencia artificial como apoyo para la estructuración del informe y la redacción de algunas secciones. Todo el código, los experimentos, las capturas y el análisis de resultados fueron realizados y comprendidos por mí. Soy capaz de explicar y defender cada parte del trabajo entregado.

Anexo C- Urls de comprobación

- **La URL del repositorio de GitHub:**
  https://github.com/Erigudis/SistemasEmbebidos
- **La URL del video de YouTube:**
  https://www.youtube.com/watch?v=coPr7ewjFDc